

Cryptomania Exchange Pro Documentation

Updated project documentation with current local technology requirements.

Technology used

- Php 8.3
- Laravel 13
- MySQL 8.4
- Apache 2
- AdminLTE 4 for the administrative panel

Introduction

Introduction

Cryptomania Exchange Pro is built with laravel. All the codes supported by laravel and php will also work here. It is a trading platform to build a super fast cryptocurrency trading platform for business

Technology used

- Php 8.3
- MySql 5.7+
- Laravel 5.6+
- Cache Driver File
- Redis Server
- Laravel Echo Server
- Supervisor

Features

- Coinpayments Coin Api Integrated
- BTC node Integrated
- Exchange Scroller
- Instant Payment notice (IPN) Ready
- Unlimited Coin adding ability
- Unlimited Coin Pair Creation
- CryptoCoin Creation
- Withdrawal and deposit histories
- Wallet report
- Order report
- Buy, Sell, Stop Limit
- Live candlestick chart
- Moving average lines
- Live MACD Graph
- Exchange report
- Google 2FA Authentication
- Google Recaptcha
- ID verification
- Detail admin report
- Dynamic role management system
- Dynamic navigation system
- Admin and User notification
- Mobile Ready

- Ultra Fast Trading Algorithm
- Easy to go User Experience

File Structure

The default Laraframe application structure is intended to provide a great starting point for both large and small applications. But you are free to organize your application however you like. Laraframe imposes almost no restrictions on where any given class is located - as long as Composer can autoload the class.

The Bootstrap Directory

The ``bootstrap`` directory contains the ``app.php`` file which bootstraps the framework. This directory also houses a cache directory which contains framework generated files for performance optimization such as the route and services cache files.

The Config Directory

The ``config`` directory, as the name implies, contains all of your application's configuration files. It's a great idea to read through all of these files and familiarize yourself with all of the options available to you.

The Database Directory

The ``database`` directory contains your database migrations, model factories, and seeds. If you wish, you may also use this directory to hold an SQLite database.

The Public Directory

The ``public`` directory contains the `index.php` file, which is the entry point for all requests entering your application and configures autoloading. This directory also houses your assets such as images, JavaScript, and CSS.

The Resources Directory

The ``resources`` directory contains your views as well as your raw, un-compiled assets such as LESS, SASS, or JavaScript. This directory also houses all of your language files.

The Routes Directory

The ``routes`` directory contains all of the route definitions for your application.

Inside ``routes/groups`` directory you can see additional route files.

The ``guest_permission.php`` and ``guest_permission_api.php`` file contains routes that anyone can access without any permission or login.

The ``permission.php`` and ``permission_api.php`` file contains routes that only can access logged in and permitted user,

The ``verification_permission.php`` and ``verification_permission_api.php`` file where defined all of your verification related routes.

The ``web.php`` file where you may define any routes. But we recommend to define routes inside ``guest_permission.php`` and ``permission.php``.

The ``api.php`` file where you may define any API routes. But we recommend to define routes inside ``guest_permission_api.php`` and ``permission_api.php``

The ``console.php`` file is where you may define all of your Closure based console commands.

The ``channels.php`` file is where you may register all of the event broadcasting channels that your application supports.

The Storage Directory

The ``storage`` directory contains your compiled Blade templates, file based sessions, file caches, and other files generated by the framework. This directory is segregated into ``app``, ``framework``, and ``logs`` directories. The ``app`` directory may be used to store any files generated by your application. The ``framework`` directory is used to store framework generated files and caches. Finally, the ``logs`` directory contains your application's log files.

The ``storage/app/public`` directory may be used to store user-generated files, such as profile avatars, that should be publicly accessible. You should create a symbolic link at ``public/storage`` which points to this directory. You may create the link using the ``php artisan storage:link`` command.

The Tests Directory

The ``tests`` directory contains your automated tests.

The Vendor Directory

The vendor directory contains your Composer dependencies.

The App Directory

The majority of your application is housed in the app directory. By default, this directory is namespaced under App and is autoloaded by Composer using the PSR-4 autoloading standard.

A variety of other directories will be generated inside the app directory as you use the make Artisan commands to generate classes. So, for example, the `app/Jobs` directory will not exist until you execute the `make:job` Artisan command to generate a job class.

Many of the classes in the app directory can be generated by Artisan via commands. To review the available commands, run the `php artisan list make` command in your terminal.

The Broadcasting Directory

The ``Broadcasting`` directory contains all of the broadcast channel classes for your application. These classes are generated using the ``make:channel`` command. This directory does not exist by default, but will be created for you when you create your first channel. To learn more about channels, check out the documentation on event broadcasting.

The Console Directory

The ``Console`` directory contains all of the custom Artisan commands for your application. These commands may be generated using the ``make:command`` command. This directory also houses your console kernel, which is where your custom Artisan commands are registered and your scheduled tasks are defined.

The Events Directory

This directory does not exist by default, but will be created for you by the ``event:generate`` and ``make:event`` Artisan commands. The ``Events`` directory houses event classes. Events may be used to alert other parts of your application that a given action has occurred, providing a great deal of flexibility and decoupling.

The Exceptions Directory

The ``Exceptions`` directory contains your application's exception handler and is also a good place to place any exceptions thrown by your application. If you would like to customize how your exceptions are logged or rendered, you should modify the ``Handler`` class in this directory.

The Http Directory

The ``Http`` directory contains your controllers, middleware, and form requests. Almost all of the logic to handle requests entering your application will be placed in this directory.

The Jobs Directory

This directory does not exist by default, but will be created for you if you execute the ``make:job`` Artisan command. The ``Jobs`` directory houses the queueable jobs for your application. Jobs may be queued by your application or run synchronously within the current request lifecycle. Jobs that run synchronously during the current request are sometimes referred to as "commands" since they are an implementation of the command pattern.

The Listeners Directory

This directory does not exist by default, but will be created for you if you execute the ``event:generate`` or ``make:listener`` Artisan commands. The ``Listeners`` directory contains the classes that handle your events. Event listeners receive an event instance and perform logic in response to the event being fired. For example, a ``UserRegistered`` event might be handled by a ``SendWelcomeEmail`` listener.

The Mail Directory

This directory does not exist by default, but will be created for you if you execute the ``make:mail`` Artisan command. The ``Mail`` directory contains all of your classes that represent emails sent by your application. Mail objects allow you to encapsulate all of the logic of building an email in a single, simple class that may be sent using the ``Mail::send`` method.

The Notifications Directory

This directory does not exist by default, but will be created for you if you execute the ``make:notification`` Artisan command. The ``Notifications`` directory contains all of the "transactional" notifications that are sent by your application, such as simple notifications about events that happen within your application. Notification features abstracts sending notifications over a variety of drivers such as email, Slack, SMS, or stored in a database.

The Policies Directory

This directory does not exist by default, but will be created for you if you execute the ``make:policy`` Artisan command. The ``Policies`` directory contains the authorization policy classes for your application. Policies are used to determine if a user can perform a given action against a resource. For more information, check out the authorization documentation.

The Providers Directory

The ``Providers`` directory contains all of the service providers for your application. Service providers bootstrap your application by binding services in the service container, registering events, or performing any other tasks to prepare your application for incoming requests.

In a fresh Laravel application, this directory will already contain several providers. You are free to add your own providers to this directory as needed.

The Rules Directory

This directory does not exist by default, but will be created for you if you execute the ``make:rule`` Artisan command. The ``Rules`` directory contains the custom validation rule objects for your application. Rules are used to encapsulate complicated validation logic in a simple object. For more information, check out the validation documentation.

The Models Directory

The ``Models`` directory contains your all of your Eloquent Model. To make a new Eloquent Model execute the ``make:model`` artisan command. But don't forget to append ``Models`` directory before define your new Eloquent Model name like ``php artisan make:model Models/NewModelName``. Otherwise model will generate inside your ``app`` directory.

The Repositories Directory

The ``Repositories`` directory contains your Eloquent Model Repository and Interface. To make a repository and interface execute ``make:repository`` and ``make:interface`` Artisan command. This will create a Repository Class and Interface inside your ``Repositories`` directory. After creating Repository and Interface you need to bind your Repository with Interface in ``RepositoryServiceProvider``.

The Services Directory

The ``Services`` directory contains your Service Class. To make aa new Service execute ``make:service`` Artisan command. This will create a Service class inside your ``Services`` directory.

Installation

Prerequisites

- Linux Server (Preferred Ubuntu 18.04)
- RAM 4GB
- Space 40GB
- PHP 8.3
- Database (Preferred MYSQL 5.7+)
- Apache 2.4/Nginx Server (Preferred Apache 2.4)
- Process Control System (Preferred Supervisord)
- PHP Dependency Manager (Preferred Composer)
- Redis Server For Queue Management
- Node Package Manager (Preferred NPM)

Prerequisites PHP extension

- OpenSSL PHP Extension
- PDO PHP Extension
- Mbstring PHP Extension
- Tokenizer PHP Extension
- XML PHP Extension
- Ctype PHP Extension
- JSON PHP Extension
- Curl PHP extension
- GD PHP extension
- Imagick PHP extension
- Intl PHP extension
- BCMath PHP Extension
- ZIP PHP extension
- UNZIP PHP extension

Installation

Cryptomania Exchange Pro is made on Laravel and installation system is same as Laravel. See Laravel Installation Guide

N.B: Before installing Cryptomania Exchange Pro make sure all prerequisites software and prerequisites php extension are installed properly.

Installing Cryptomania Exchange Pro

Cryptomania Exchange Pro utilizes Composer to manage its dependencies. So, before using Cryptomania Exchange Pro, make sure you have Composer installed on your machine.

Unzip Cryptomania Exchange Pro project and put it into your server directory (For linux the server directory is `/var/www/html/`). After this go to your project directory and run ``composer install`` to download all the dependency.

Directory Permissions

After installing Cryptomania Exchange Pro, you may need to configure some permissions. Directories within the `storage` and the `bootstrap/cache` directories should be writable by your web server or Cryptomania Exchange Pro will not run.

example (for ubuntu. may vary for other linux distributions):

```
sudo chown -R www-data:www-data bootstrap/cache
sudo chown -R www-data:www-data storage
```

Environment Setup

The next thing you should do after installing Cryptomania Exchange Pro is Renaming the `.env.example` file to `.env`, you should do that now.

```
APP_NAME="My App" # Your own exchange name
APP_ENV=local # Change it to production once installation is done
APP_KEY= # This will be generated by the command given below
APP_DEBUG=true # Must be false in production mode
APP_PROTOCOL=http # Which protocol you are using
APP_URL=http://dev.crypto.exchange # Your Web Url
APP_TIMEZONE= # Set it to 'UTC' or any timezone you prefer
```

```
LOG_CHANNEL=stack
```

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=homestead # Your database name
DB_USERNAME=homestead # Your mysql username
DB_PASSWORD=secret # Your mysql password
```

```
BROADCAST_DRIVER=pusher
CACHE_DRIVER=file
SESSION_DRIVER=file
SESSION_LIFETIME=120
QUEUE_DRIVER=redis
```

```
REDIS_HOST=127.0.0.1
REDIS_PASSWORD=null
REDIS_PORT=6379
```

```
# Your Email Configuration
```

```
MAIL_DRIVER=smtp
MAIL_HOST=smtp.mailtrap.io
MAIL_PORT=2525
MAIL_USERNAME=null
MAIL_PASSWORD=null
MAIL_ENCRYPTION=null
MAIL_FROM_ADDRESS= # The sender email address
MAIL_FROM_NAME= # The sender name
```

```
# Your coinpayment Configuration (see coinpayment section for details)
```

```
COINPAYMENT_PRIVATE_KEY=
COINPAYMENT_PUBLIC_KEY=
COINPAYMENT_MERCHANT_ID=
COINPAYMENT_IPN_SECRET=
COINPAYMENT_IPN_URL=http://www.example.com/api/ipn
```

```
PAYPAL_CLIENT_ID=
PAYPAL_SECRET=
PAYPAL_INTENT=
RETURN_URL=
CANCEL_URL=
PAYPAL_MODE=
```

```
BTC_API_SCHEME=
BTC_API_HOST=
```

```
BTC_API_PORT=  
BTC_API_RPCUSER=  
BTC_API_RPCPASSWORD=  
BTC_API_NETWORK_FEE=  
BTC_API_SSL_CERT=
```

```
NOCAPTCHA_SECRET=secret-key  
NOCAPTCHA_SITEKEY=site-key
```

Application Key

Set your application key to a random string. Typically, this string should be 32 characters long. The key can be set in the `.env` environment file. If the application key is not set, your user sessions and other encrypted data will not be secure! Follow the command below to generate the key

```
php artisan key:generate
```

Additional Configuration

By default Cryptomania Exchange Pro store cache and session in files. If you want to change it rename cache and session driver from `.env` and clear config cache by running `php artisan config:clear` otherwise the change will not affect on your application.

Install Database

To make your mysql database ready you need to run following command.

```
php artisan migrate
```

Install initial Data

To push initial data in database, run the following command.

```
php artisan db:seed
```

N:B: The website won't run without initial data. But before going live, You must change seeds for ``users``, ``userinfos``, ``usersettings``, ``coinitems`` and ``coinpairs``. You may remove some data as well if needed. It is very important to keep ``superadmin`` in ``users``, ``userinfos``, ``usersettings`` seeders. But you must change the ``username``, ``email`` and ``password`` of ``superadmin`` to keep it secured. Most importantly do not remove anything from ``adminsettings`` seeder.

To refresh the initial data in database after changing the seeders, run the following command (do not do it once you go live)

```
php artisan migrate:fresh --seed
```

Supervisor & Horizon Configuration

Installing Supervisor

Supervisor is a process monitor for the Linux operating system, and will automatically restart your `queue:work` process if it fails. To install Supervisor on Ubuntu, you may use the following command:

```
sudo apt-get install supervisor
```

If configuring Supervisor yourself sounds overwhelming, consider using Laravel Forge, which will automatically install and configure Supervisor for your Laravel projects.

Supervisor configuration files are typically stored in the `/etc/supervisor/conf.d` directory. Within this directory, you may create any number of configuration files that instruct supervisor how your processes should be monitored. For example, let's create a `horizon.conf` file that starts and monitors a `horizon` process:

```
[program:horizon]
process_name=%(program_name)s
command=php /var/www/html/cryptoexchange/artisan horizon
autostart=true
autorestart=true
user=root
redirect_stderr=true
stdout_logfile=/var/log/horizon.log
```

Replace `cryptoexchange` to your project directory name. If your server does not have `root` user, then change the `user` as well. Make sure `/var/log/horizon.log` the file is created and has write permission

Starting Supervisor

Once the configuration file has been created, you may update the Supervisor configuration and start the processes using the following commands:

```
sudo supervisorctl reread
```

```
sudo supervisorctl update
```

```
sudo supervisorctl start horizon
```

You need to run the following command to check if your supervisor is running successfully.

```
sudo supervisorctl status
```

Real Time Data Configuration

You can configure real time data in two ways in Cryptomania Exchange Pro.

- Pusher (Easy setup, no hassle)
- Laravel Echo Server

Pusher Configuration

Please go to https://dashboard.pusher.com/accounts/sign_up and open an account if you don't have any. After logging in your account please create a new app for your exchange. Once creation is done you will be redirect to a page where `app keys` tab will have all the credential you need. Copy them and paste them under the related keys in `.env` file. check the setting here. You will get the following keys there.

```
PUSHER_APP_ID=  
PUSHER_APP_KEY=  
PUSHER_APP_SECRET=  
PUSHER_APP_CLUSTER=
```

Open `/var/www/html/resources/assets/js/bootstrap.js` file and replace line 9 to 16 with bellow code

```
window.Pusher = require('pusher-js');  
  
window.Echo = new Echo({  
  broadcaster: 'pusher',  
  key: process.env.MIX_PUSHER_APP_KEY,  
  cluster: process.env.MIX_PUSHER_APP_CLUSTER,  
  encrypted: false  
});
```

After changing any configuration run following commands:

```
npm run production //'npm run dev' for development  
php artisan clear:all  
php artisan horizon:terminate
```

Laravel Echo Server Configuration

- Run `npm install -g laravel-echo-server` command to install laravel echo server globally.
- Make sure `/var/www/html/resources/assets/js/bootstrap.js` file line 9 to 16 has the following code

```
window.io = require('socket.io-client');  
  
window.Echo = new Echo({  
  broadcaster: 'socket.io',  
  host: window.location.hostname + ':6001'  
});
```

- After installing run `npm run dev` or `npm run production` depending on your environment.
- Open laravel-echo-server.json file and modify the following variables

```
"authHost" : "http://your_site_url",  
"port" : "6001",  
"protocol" : "http",
```

If use `https` then you need to change `protocol` to `https` and add your ssl certificate and private key absolute path. If you don't upload these file into server then upload it first.

```
"sslCertPath": "ssl.cert",  
"sslKeyPath": "ssl.key",
```

- Run `php artisan clear:all` to clear all cache.

Configure Laravel Echo Server in Supervisor

- Create `echo-server.conf` config file inside `/etc/supervisor/conf.d` this directory.
- Create `echo-server.log` file inside `/var/log` directory and give write permission.
- Open `echo-server.conf` and paste the following configuration and save.

```
[program:echo-server]
process_name=%(program_name)s
command=laravel-echo-server start --dir=/var/www/html/
autostart=true
autorestart=true
user=root
redirect_stderr=true
stdout_logfile=/var/log/echo-server.log
```

Starting Laravel Echo Server

Once the configuration file has been created, you may update the Supervisor configuration and start the processes using the following commands:

```
sudo supervisorctl reread
sudo supervisorctl update
sudo supervisorctl start echo-server
```

You need to run the following command to check if your supervisor is running successfully.

```
sudo supervisorctl status
```

Quick Start Guide

Login

Login with username/email and password.

Demo Users

username password superadmin superadmin trader trader

Register

Register with required information.

Profile Control

control your profile info, password, Google 2FA, Avatar and Id verification.

Role Management

edit existing role access or create a new role with selected access.

Admin Setting

edit settings from Admin setting section to control your exchange App.

Coin Management

edit or create a coin with required fields. Api service must be selected

Coin Pair Management

edit or create a coinpair with required fields.

Coinpayment Configuration

You can see our video tutorial for coinpayments configuration: [Coinpayments Configuration](#).

Create Coinpayments Account

Please go to <https://www.coinpayments.net> and open an account if you don't have any. After logging in your account please you need to collect the following credentials -

Private And Public Key

To generate private and public keys go to this `API Keys` tab. You can find this tab under `Account > API KEY` or visit this link <https://www.coinpayments.net/acct-api-keys> . Click on the `Generate new key...` Button and you will get public and private keys. Copy and paste them in your .env file.

Next click on the `Edit Permissions` button and you will be have two sections, one is `API Key Settings` and another `API Key Permissions`.

You can configure them according to your requirements. For `API Key Permission` please check all the options except `get\_basic\_info` and `set\_basic\_info`.

Coinpayments Merchant ID

Please visit `Account Settings` under `Account` menu or click on this <https://www.coinpayments.net/acct-settings> and you will get your Merchant ID under `Basic Settings` tab. Please copy and set this as value for `COINPAYMENT\_MERCHANT\_ID` in `.env` file.

Coinpayments IPN Secret

Please visit `Account Settings` under `Account` menu or click on this <https://www.coinpayments.net/acct-settings> and select `Merchant Settings` tab. The very first field is for `IPN secret` and input random string and save.

Use the same `IPN secret` value for `COINPAYMENT\_IPN\_SECRET` in `.env` file.

Coinpayments IPN Url

Set value as `http://yourdomainname.com/api/ipn` in `.env` file. The end point must be `/api/ipn`.

Check the environment setting process here.

After changing any configuration run following commands:

```
php artisan clear:all
php artisan horizon:terminate
```

PayPal Configuration

Cryptomania Exchange PRO provides PayPal API configuration for interacting with PayPal payment method. To enable PayPal Api the following options need to be configured in project .env file.

```
PAYPAL_CLIENT_ID=  
PAYPAL_SECRET=  
PAYPAL_INTENT=  
RETURN_URL=  
CANCEL_URL=  
PAYPAL_MODE=
```

All fields are required except `RETURN\_URL` and `CANCEL\_URL`.

NB: Change PAYPAL_MODE value to live for production mode. After setting all the required option you need to clear cache by running this command `php artisan clear:all` and `php artisan horizon:terminate` to clear queue cache

To Obtain API Client ID And Secret Key Within Your Paypal Account -

- Please go to <https://developer.paypal.com> and Log In
- Go to the dashboard in this website.
- Click on Create App Button
- Fill out the information "Create New App" this page.
- Here you will get your REST API Credentials Client id and Secret Key.

For more information check here :- <https://developer.paypal.com/docs/api/overview/#get-credentials>

BTC API Configuration

Cryptomania Exchange PRO provides BTC API for interacting with BTC node. To enable BTC Api the following options need to be configured in project .env file.

```
BTC_API_SCHEME=  
BTC_API_HOST=  
BTC_API_PORT=  
BTC_API_RPCUSER=  
BTC_API_RPCPASSWORD=  
BTC_API_NETWORK_FEE=
```

After setting the above options run following commands:

```
php artisan clear:all  
php artisan horizon:terminate
```

To active BTC wallet notify you may add the following url into your BTC node server
`http://example.com/api/bitcoin/ipn/BTC`